



UNIVERSITY OF SCIENCE - VNUHCM

FACULTY OF INFORMATION TECHNOLOGY

SOFTWARE TESTING

HW6 - PERFORMANCE TESTING

Software Testing Project Report

Authors:

Cao Uyển Nhi (22127310)
cunhi22@clc.fitus.edu.vn

Supervisors:

Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

August 12, 2025

Table of Contents

1	Group Information	1
2	Server / PC Configuration	1
3	Performance Testing Techniques Applied	1
3.1	Test case 1: Load Testing	1
3.2	Test case 2: Spike Testing	2
3.3	Test case 3: Stress Testing	2
4	Data-Driven Testing Implementation	3
5	Recording Test Scripts	3
6	Applying AI in Performance Testing	4
6.1	Prompt 1 — Scenario Design	4
6.2	Prompt 2 — Groovy Randomization Logic	4
6.3	Prompt 3 — Debugging URI Errors	5
6.4	Prompt 4 — Adding Think Time	5
6.5	Prompt 5 — Spike & Stress Test Configuration	5
7	Youtube Video Link	5
8	Self-Assessment	5

1 Group Information

Group ID: 07

Member Name	Student ID	Assigned Features	Status
Cao Uy��n Nhi	22127310	Homepage - Category - Product	Done
L��u Thanh Thu��y	22127410	Register	Done
Nguy��n Ph����c Minh Tr��	22127424	Homepage - Sort - Search	Done
V�� L�� Vi��t T��	22127435	Sign In - Invoice	Done
Tr��n Th��i C��t T����ng	22127444	Homepage - Contact	Done

2 Server / PC Configuration

Component	Specification
OS	macOS Sonoma 14.x
CPU	Apple M1, 8 cores
RAM	16 GB
Disk	512 GB SSD
JMeter Version	Apache JMeter 5.6.x
Java Version	OpenJDK 17
Browser	Chrome (for script recording)
Website Under Test	The Toolshop (Localhost, Angular front-end + backend API)

3 Performance Testing Techniques Applied

I applied **three types of performance tests**:

- **Load Testing** – Determine if the application can handle the expected number of concurrent users under normal conditions.
- **Spike Testing** – Measure the system’s behavior under sudden and extreme load increases.
- **Stress Testing** – Identify the breaking point and observe how the system behaves beyond normal operational capacity.

3.1 Test case 1: Load Testing

Objective: Verify whether the system can handle the expected normal traffic of **50 concurrent users** without performance degradation.

User Journey:

1. Access the home page (/)
2. Select a category (randomized from `category.csv`)
3. Select a product (randomized from `product.csv`)

4. View the product details page

Thread Group Configuration:

Parameter	Value
Number of Threads	50
Ramp-up Period	250 sec
Loop Count	Infinite (or fixed for short test)
Assertions	Response time $\leq$ 2000 ms, HTTP status = 200

Expected Result:

- Stable response times within 2 seconds
- Zero or negligible error rate

3.2 Test case 2: Spike Testing

Objective: Assess the system's reaction and recovery time when traffic suddenly increases.

User Journey: Same as in the Load Testing scenario.

Ultimate Thread Group Configuration:

Stage	Virtual Users	Ramp-up	Hold Load	Shutdown
Warm-up	10	30 sec	60 sec	5 sec
Spike	100	30 sec	60 sec	5 sec
Recovery	10	10 sec	60 sec	5 sec

Expected Result:

- Slight increase in response times during spike, but within 3000 ms
- Near-zero error rate
- System returns to normal performance after load decreases

3.3 Test case 3: Stress Testing

Objective: Identify the maximum load capacity and observe behavior beyond the system's limits.

User Journey: Same as in the Load Testing scenario.

Ultimate Thread Group Configuration:

Stage	Virtual Users	Ramp-up	Hold Load	Shutdown
1	60	10 sec	60 sec	5 sec
2	80	10 sec	60 sec	5 sec
3	120	10 sec	60 sec	5 sec
4	160	10 sec	60 sec	5 sec

Stage	Virtual Users	Ramp-up	Hold Load	Shutdown
5	200	10 sec	120 sec	10 sec

Expected Result:

- Response time and error rate noticeably increase when load exceeds ~180 virtual users
- At 200 virtual users, the system shows instability or failures

4 Data-Driven Testing Implementation

To simulate realistic and varied user behavior, all performance test scripts applied **data-driven techniques** in JMeter.

This prevents all virtual users from executing the exact same requests and helps uncover performance issues that may only appear with diverse input data.

Two CSV files were created based on actual data from **The Toolshop** database:

File Name	Columns	Example Row
<code>categories.csv</code>	<code>category_id, category_slug</code>	<code>C001,hand-tools</code>
<code>products.csv</code>	<code>product_id, category_id, product_slug</code>	<code>P102,C001,hammer-pro-16oz</code>

- **categories.csv**: Stores the category ID and slug for each product category.
- **products.csv**: Stores the product ID, its category ID, and the product slug.

For each test script:

- Add a **CSV Data Set Config** element to read from `categories.csv` and/or `products.csv`.
- Example configuration:
 - **Filename**: `${_P(testdata_dir, )}/categories.csv`
 - **Variable Names**: `category_id,category_slug`
 - **Recycle on EOF?**: True
 - **Stop thread on EOF?**: False
 - **Sharing mode**: All threads

5 Recording Test Scripts

All three test scripts (Load, Spike, Stress) were initially created using JMeter's **HTTP(S) Test Script Recorder** to capture the exact user journey in **The Toolshop** application.

- **Start the Recorder** in JMeter.
- In Chrome, perform the test scenario steps:
 - Open the home page /
 - Navigate to a category (using sample data)
 - View a product list

- Open product details
- Stop recording when the full journey is completed.

6 Applying AI in Performance Testing

During the performance testing process, AI tools were utilized to enhance efficiency, accuracy, and reporting quality. The key applications include:

6.1 Prompt 1 — Scenario Design

Objective: Generate realistic performance testing scenarios for The Toolshop with limited database accounts.

Prompt Example: “When conducting performance testing, what scenarios can be created for The Toolshop given that user registration is not possible and only two accounts exist in the database?”

AI Output:

Suggested focusing on **view-only scenarios**:

- Browse categories
- View products in category
- View product details

Recommended adding load patterns for:

- Load Testing
- Spike Testing
- Stress Testing

6.2 Prompt 2 — Groovy Randomization Logic

Objective: Automate random category and product selection from CSV data for JMeter scripts.

Prompt Example: “Write a JSR223 (Groovy) script in JMeter that reads `categories.csv` and `products.csv`, picks a random category slug, and then picks a random product in that category.”

AI Output:

Provided Groovy code using:

- `FileReader` + `CSVParser` to read files
- Filtering products by category ID
- Storing selected values in JMeter variables (`vars.put("cat_slug", ...)`, `vars.put("product_id", ...)`)

Advised **not to use Random Controller** if randomization is already handled in Groovy.

6.3 Prompt 3 — Debugging URI Errors

Objective: Fix `java.net.URISyntaxException` when using JMeter with dynamic URLs.

Prompt Example: “I get Illegal character in fragment for `http://localhost:4200/#/category` in JMeter. How to fix?”

AI Output:

- Explained that `#` fragments are not sent to the server and must be handled differently.
- Suggested switching to actual REST API endpoints instead of SPA routes for backend performance measurement.

6.4 Prompt 4 — Adding Think Time

Objective: Simulate realistic user behavior with pauses between requests.

Prompt Example: “How to add think time between requests in JMeter without breaking my randomization logic?”

AI Output:

- Recommended **Constant Timer** or **Uniform Random Timer** after HTTP Samplers.
- Suggested placing timers inside the Thread Group or directly after each request for scenario realism.

6.5 Prompt 5 — Spike & Stress Test Configuration

Objective: Tune JMeter parameters for different performance testing types.

Prompt Example: “Given my load test uses 50 users, 30s ramp-up, 1 min hold, what should I set for spike testing and stress testing?”

AI Output:

- Spike Test: Sudden jump to peak load (e.g., $0 \rightarrow 100$ users in 5s), short hold.
- Stress Test: Gradual ramp beyond expected limits (e.g., +20 users every 30s until failure).

7 Youtube Video Link

[Load Testing Process](#)

[Spike Testing Process](#)

[Stress Testing Process](#)

8 Self-Assessment

Criteria	Description	Max Points	Self Assessment
Load testing	Report: 1.0 TestCases, BugReport: 0.5 Script, Data: 0.5 Video: 1.0	3.0	3.0
Stress testing	Report: 1. TestCases, BugReport: 0.5 Script, Data: 0.5 Video: 1.0	3.0	3.0
Spike testing	Report: 1. TestCases, BugReport: 0.5 Script, Data: 0.5 Video: 1.0	3.0	3.0
Use of AI Tools	Prompt transparency, critical validation, added value	1.0	1.0
Total		10.0	10.0